Enhanced Pareto Local Search for the Satellite Image Mosaic Selection Problem: Greedy Initialisation, Checkpoint-Based Tracking, and Perturbation Restart

Anonymous Authors

Abstract

The Satellite Image Mosaic Selection (SIMS) problem requires choosing a cost-effective set of satellite images that jointly cover a geographic area of interest while simultaneously minimising acquisition cost, cloud coverage, spatial resolution loss, and worst-case incidence angle. We present three complementary enhancements to the Pareto Local Search (PLS) heuristic that serves as the metaheuristic engine inside a hybrid exact/heuristic solver framework.

First, we introduce a *greedy multi-objective initialisation* strategy that seeds the search with diverse, high-quality feasible solutions constructed via weighted greedy set cover using $\binom{D}{1}+\binom{D}{2}+1$ weight vectors, where D is the number of objectives. Second, we propose a *two-level checkpoint mechanism* for the incremental objective trackers that replaces repeated per-image undo operations with bulk state restoration, yielding constant-time state transitions during neighbourhood evaluation. Third, we design a *perturbation-based restart* that, when the k=1 neighbourhood is exhausted, injects randomly perturbed archive solutions back into the working population, providing diversification without the combinatorial expense of higher-order neighbourhoods.

Experiments across 20 benchmark instances (5 cities $\times$ 4 sizes, 30–150 images, 4 objectives) show that the combined enhancements improve normalised hypervolume by up to +57.8% and by +11.7% on average over the baseline PLS, with the largest gains on instances where the baseline converges prematurely. Hypervolume-over-time curves confirm that the improvements manifest within the first seconds of execution and persist throughout the run.

Keywords. multi-objective optimisation, Pareto local search, satellite image selection, set cover, hypervolume, greedy initialisation, perturbation restart

1 Introduction

Earth observation planning increasingly relies on automated methods to select mosaics of satellite images that cover a region of interest (ROI) while balancing conflicting quality criteria. The resulting *Satellite Image Mosaic Selection* (SIMS) problem is a multi-objective variant of the weighted set cover problem and is known to be NP-hard.

In a companion study we developed a *two-phase hybrid* solver that allocates part of the time budget to an exact method (AUGMECON/GPBA-based MILP) and the remainder to a Pareto Local Search (PLS) metaheuristic. The exact phase produces a small set of provably efficient anchor solutions; PLS then densifies the Pareto front around and between them. The quality of the final approximation set depends critically on the effectiveness of PLS, which in turn depends on (i) the quality of its starting population, (ii) the throughput of its neighbourhood evaluation, and (iii) its ability to escape local optima.

Contributions. We identify three independent weaknesses of the baseline PLS and propose targeted fixes:

1. **Greedy multi-objective initialisation** (Section 4): replaces the random initial population with diverse feasible solutions constructed by a weighted greedy set-cover heuristic, one per objective axis, per objective pair, and one balanced, giving $\binom{D}{1} + \binom{D}{2} + 1$ anchor points.
2. **Two-level checkpoint mechanism** (Section 5): introduces $O(1)$ bulk state restore for incremental objective trackers, eliminating redundant per-image undo operations during neighbourhood enumeration.
3. **Perturbation restart** (Section 6): when the working population's k=1 neighbourhood is exhausted, randomly perturbed copies of archive solutions are injected, resetting the neighbourhood level and avoiding the combinatorial cost of $k \geq 2$.

Each technique is independently toggleable, enabling controlled ablation. We evaluate all combinations on 15 instances with D=4 objectives and report hypervolume improvements of up to 60%.

2 Problem Formulation

2.1 Decision Space

An instance of SIMS comprises a universe $U = \{1, \ldots, m\}$ of geographic segments (elements), a catalogue $\mathcal{I} = \{1, \ldots, n\}$ of candidate satellite images, and for each image i the set $S_i \subseteq U$ of elements it covers. A *feasible mosaic* is a subset $X \subseteq \mathcal{I}$ such that $\bigcup_{i \in X} S_i = U$, i.e. a *set cover*. Instance sizes in our benchmark range from n=30, m=298 to n=200, m=25 271.

2.2 Objective Space

All four objectives are to be *minimised*:

$$f_1(X) = \sum_{i \in X} c_i \qquad \text{(total acquisition cost)}, \tag{1}$$

$$f_2(X) = \sum_{e \in U} a_e \cdot \mathbf{1}\big[\forall\, i \in X :\ e \notin C_i\big] \qquad \text{(cloudy area)}, \tag{2}$$

$$f_3(X) = \sum_{e \in U} \min_{i \in X:\, e \in S_i} r_i \qquad \text{(resolution sum)}, \tag{3}$$

$$f_4(X) = \max_{i \in X} \alpha_i \qquad \text{(worst incidence angle)}, \tag{4}$$

where c_i is the per-image cost, a_e is the area of element e, $C_i \subseteq S_i$ denotes the *clear* elements of image i (complement of cloud mask within coverage), r_i is the spatial resolution of image i, and α_i its incidence angle.

Objectives (1)–(3) are *additive or element-wise aggregations* over the selected images, while (4) is a *bottleneck* (max) objective. This structural heterogeneity has important consequences for neighbourhood design and incremental evaluation.

3 Baseline PLS

Our starting point is a population-based PLS with the classic *accept non-dominated neighbours* acceptance criterion and a *variable neighbourhood structure* $k \in \{1, \ldots, k_{\max}\}$.

3.1 Neighbourhood Operator

The neighbourhood $\mathcal{N}_k(X)$ of a feasible solution X is defined by a *remove-then-repair* scheme:

Algorithm 1 Baseline Pareto Local Search

Input: problem instance $(U, \mathcal{I}, f_1, \ldots, f_D)$, timeout T, $k_{\max}$
Output: approximate Pareto set $\mathcal{A}$

$P \leftarrow$ random feasible solutions; $\mathcal{A} \leftarrow P$; $k \leftarrow 1$
while elapsed time $< T$ **do**
  $P_{\text{aux}} \leftarrow \emptyset$
  for each $X \in \textsc{Shuffle}(P)$ **do**
    for each $X' \in \mathcal{N}_k(X)$ **do**
      if X' not previously explored **and** X' is non-dominated w.r.t. $\mathcal{A}$ **then**
        $\mathcal{A}.\textsc{Insert}(X')$; $P_{\text{aux}}.\textsc{Insert}(X')$
  if $P_{\text{aux}} \neq \emptyset$ **then**
    $P \leftarrow P_{\text{aux}}$; $k \leftarrow 1$ ▷ restart with new population
  else if $k < k_{\max}$ **then**
    $k \leftarrow k + 1$; add eligible archive solutions to P
  else
    break ▷ all neighbourhood structures exhausted
return $\mathcal{A}$

1. **Remove** k selected images from X, producing a partial solution $X' = X \setminus R$ with $|R| = k$.
2. **Identify** uncovered elements $U' = U \setminus \bigcup_{i \in X'} S_i$.
3. **Repair**: select a *condensed* sub-problem of the ℓ best addition candidates (ranked by a random weighted scalarisation of objective deltas) and enumerate all subsets that restore coverage, yielding a set of feasible neighbours.

For k=1, every selected image that is *replaceable* (at least one unselected image overlaps with it) is a removal candidate. For k>1, the algorithm selects the q=9 worst-scoring images and iterates over $\binom{q}{k}$ removal combinations.

3.2 Incremental Objective Trackers

Evaluating objectives from scratch for each neighbour is prohibitively expensive. Instead, four *incremental trackers* maintain mutable state that is updated in $O(|S_i|)$ time per image addition or removal:

- **TotalCost:** a running scalar sum; $O(1)$ delta.
- **CloudyArea:** per-element counters `clear_count`$[e]$; delta when counter transitions $0 \leftrightarrow 1$.
- **MinResolution:** per-element packed level-count arrays tracking the minimum resolution among covering images; $O(|S_i|)$ delta per add/remove.
- **MaxIncidenceAngle:** sorted level-count array of selected-image angles; $O(L)$ delta where L is the number of distinct angle values (typically $\leq$30).

3.3 Main Loop

The PLS main loop (Algorithm 1) iterates as follows. At each *step*, every solution in the current working population is explored: its neighbourhood $\mathcal{N}_k$ is enumerated, and non-dominated neighbours that have not been seen before are added to both the global Pareto archive $\mathcal{A}$ and an *auxiliary population*. If the auxiliary population is non-empty, it replaces the working population and k resets to 1. Otherwise k is incremented; if $k > k_{\max}$ the algorithm terminates.

Algorithm 2 Weighted Greedy Set-Cover Construction

Input: problem instance, weight vector $\mathbf{w} \in \mathbb{R}^D$
Output: feasible solution X

Pre-compute per-image objective proxy values $v_i^{(d)}$ for each image i and objective d:
  $v_i^{(\text{cost})} = c_i, \quad v_i^{(\text{cloud})} = |S_i \setminus C_i|, \quad v_i^{(\text{res})} = r_i, \quad v_i^{(\text{angle})} = \alpha_i$
$X \leftarrow \emptyset$; $\quad$ *covered* $\leftarrow \emptyset$
while *covered* $\neq U$ **do**
  for each unselected image $i \notin X$ **do**
    $\delta_i \leftarrow |\{e \in S_i : e \notin \textit{covered}\}|$ ▷ newly covered elements
    if $\delta_i > 0$ **then**
      $\textit{score}_i \leftarrow \left(\sum_{d=1}^{D} w_d \cdot v_i^{(d)}\right) / \delta_i$
  $i^* \leftarrow \arg\min_{i:\,\delta_i > 0} \textit{score}_i$
  $X \leftarrow X \cup \{i^*\}$; $\quad$ *covered* $\leftarrow$ *covered* $\cup\, S_{i^*}$
return X

4 Enhancement 1: Greedy Multi-Objective Initialisation

4.1 Motivation

The baseline PLS generates its initial population by constructing 100 random feasible solutions. Because the SIMS objective space is four-dimensional and the set-cover constraint is non-trivial, random solutions tend to cluster in a narrow region of the Pareto front, leaving large portions of the objective space unexplored during early iterations. Metaheuristics such as NSGA-II and MOEA/D mitigate this with population-diversity mechanisms (crowding distance, decomposition vectors); PLS has no such mechanism by default.

4.2 Method

We construct a small set of diverse, high-quality feasible solutions using a *weighted greedy set-cover* heuristic (Algorithm 2). The greedy heuristic accepts a weight vector $\mathbf{w} \in \mathbb{R}^D_{\geq 0}$ and iteratively selects the image with the best weighted score per newly covered element until coverage is complete.

To achieve broad coverage of the objective space, we invoke Algorithm 2 with a structured set of weight vectors:

1. D **axis-aligned** vectors $\mathbf{e}_d$ (weight 1 on objective d, 0 elsewhere), each producing a solution that greedily minimises one objective;
2. $\binom{D}{2}$ **pairwise** vectors with weight 0.5 on each of two objectives, covering trade-off regions between pairs;
3. 1 **balanced** vector $\mathbf{w} = (\frac{1}{D}, \ldots, \frac{1}{D})$.

For D=4 this yields $4 + 6 + 1 = 11$ greedy solutions. These are inserted into the initial population alongside the 100 random solutions and filtered through the Pareto archive, providing non-dominated anchor points in diverse regions of the front from the very first iteration.

4.3 Complexity

Each greedy construction is $O(n \cdot m)$ (at most n iterations, each scanning n images over m elements). With 11 invocations the total initialisation overhead is $O(11 \cdot n \cdot m)$, negligible compared to PLS exploration.

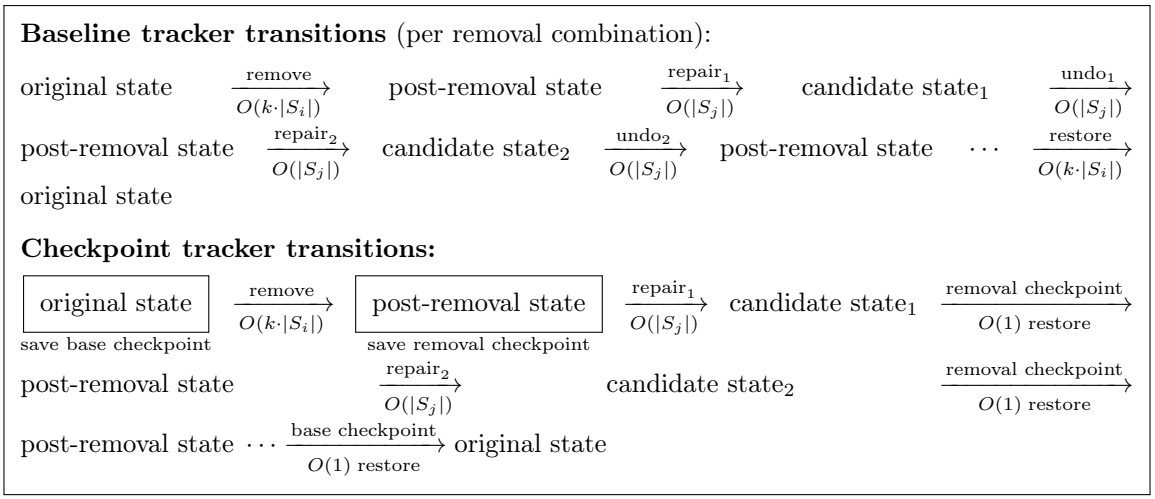

Figure 1: Tracker state transitions during neighbourhood evaluation. The checkpoint mechanism replaces iterative per-image undo with direct restoration of previously saved tracker states. The "$O(1)$" annotation refers to fixed-size state restoration whose cost depends on the universe size m but is independent of the number of images involved.

5 Enhancement 2: Two-Level Checkpoint Mechanism

5.1 Motivation

During neighbourhood evaluation, the baseline PLS modifies shared tracker state following the remove-then-repair scheme. For each removal combination, the tracker transitions through the following states:

$$\underbrace{S_{\text{base}}}_{\text{original}} \xrightarrow{\text{remove } k \text{ images}} \underbrace{S_{\text{removed}}}_{\text{partial}} \xrightarrow[\text{for each residual}]{\text{add images}} S_{\text{merged}} \xrightarrow{\text{undo}} S_{\text{removed}} \xrightarrow{\text{restore}} S_{\text{base}}$$

In the baseline, each transition is performed by iterating over individual images and calling `track_image_addition` or `track_image_removal`, each costing $O(|S_i|)$ for the CloudyArea and MinResolution trackers. With hundreds of residual solutions per removal combination and dozens of removal combinations per solution, these undo operations dominate wall-clock time.

5.2 Method

We observe that the mutable tracker state can be captured and restored as a compact snapshot. This makes it possible to replace long sequences of undo operations with direct restoration of previously saved states. We introduce *two checkpoint levels* (Figure 1):

Base checkpoint: snapshot of the original tracker state before any removals. It is used to restore the tracker when transitioning between different removal combinations.

Removed-state checkpoint: snapshot of the tracker state after a particular set of k image removals. It is used to restore the tracker after evaluating each repaired neighbour, replacing repeated per-image undo operations.

The checkpoint operation copies only the mutable portions of each tracker, leaving immutable shared data in place. In practice, the resulting snapshots are small enough that restoration remains cheap relative to repeated per-image undo operations, even on the larger instances considered in our benchmark.

5.3 Pseudocode

Algorithm 3 presents the enhanced neighbourhood evaluation procedure.

Algorithm 3 Neighbourhood Iterator with Two-Level Checkpointing

Input: solution X, neighbourhood level k, problem P
Output: stream of feasible neighbours

Initialise trackers τ from X
$C_{\text{base}} \leftarrow \textsc{Snapshot}(\tau)$ ▷ save S_{base}
for each removal combination $R \subset X$, $|R| = k$ **do**
  for each $i \in R$ **do**
    $\tau.\textsc{RemoveImage}(i)$
  $C_{\text{rem}} \leftarrow \textsc{Snapshot}(\tau)$ ▷ save S_{removed}
  $\mathcal{F} \leftarrow \textsc{EnumerateResiduals}(X \setminus R,\ P,\ \tau)$
  for each residual solution $X_r \in \mathcal{F}$ **do**
    merge X_r into $(X \setminus R)$, updating τ
    yield the merged neighbour
    $\textsc{Restore}(\tau,\ C_{\text{rem}})$ ▷ return to the post-removal state
  $\textsc{Restore}(\tau,\ C_{\text{base}})$ ▷ return to the original state

6 Enhancement 3: Perturbation-Based Restart

6.1 Motivation

When the k=1 neighbourhood of the entire working population yields no improving neighbours, the baseline PLS increments k. However, higher-order neighbourhoods ($k \geq 2$) are combinatorially expensive: the number of removal combinations grows as $\binom{q}{k}$ (with q=9), and each combination triggers a full residual enumeration. In practice, $k \geq 3$ neighbourhoods rarely find improving solutions but consume the majority of the time budget.

6.2 Method

Instead of immediately increasing k, we inject *perturbed copies* of archive solutions into the working population (Algorithm 4). Each perturbation applies a minimal structural change—removing one randomly chosen image and greedily restoring coverage—to produce a new feasible solution that is structurally close to a known Pareto-optimal solution but has not been explored before.

If at least one perturbation is successfully injected, k resets to 1 and exploration resumes with the enriched population. If no perturbations succeed (all dominated or already explored), the algorithm falls through to the standard k-increase logic.

6.3 Integration into the Main Loop

Algorithm 5 shows the complete enhanced PLS with all three improvements integrated. The additional steps relative to Algorithm 1 are highlighted for clarity.

7 Experimental Setup

7.1 Instances

We use 20 benchmark instances derived from real satellite catalogues over five cities (Lagos, Mexico City, Paris, Rio de Janeiro, Tokyo Bay) at four catalogue sizes (30, 50, 100, and 145/150 images; Lagos uses 145 images while the other four cities use 150 images at the largest tier). All instances optimise D=4 objectives simultaneously. Instance characteristics are summarised in Table 1.

Algorithm 4 Perturbation-Based Restart

Input: archive $\mathcal{A}$, problem instance, number of perturbations ρ
Output: number of injected solutions

injected $\leftarrow 0$
for each $X \in \mathcal{A}$ **do**
  for $p = 1$ **to** ρ **do**
    $i \leftarrow$ random image in X
    $X' \leftarrow X \setminus \{i\}$
    $U' \leftarrow$ uncovered elements in X'
    while $U' \neq \emptyset$ **do** ▷ greedy coverage repair
      $j^* \leftarrow \arg\max_{j \notin X'} |S_j \cap U'|$
      $X' \leftarrow X' \cup \{j^*\}; \quad U' \leftarrow U' \setminus S_{j^*}$
    Recompute all objectives of X'
    if X' is feasible **and** not previously explored **then**
      Register X' as explored
      if $\mathcal{A}$.TryInsert(X') **then** ▷ non-dominated?
        Add X' to working population P
        injected $\leftarrow$ *injected* $+ 1$
return *injected*

7.2 Configurations

We compare four configurations that isolate the contribution of the PLS enhancements and the hybrid MILP component independently:

1. **Pure PLS:** original PLS without any enhancements or hybrid component (baseline).
2. **Hybrid 50:50:** unenhanced PLS with a hybrid MILP component that uses 50% of the time budget for exact MILP solving and hands the resulting front to PLS for the remaining 50%.
3. **Improved PLS:** all three PLS enhancements enabled (greedy initialisation, two-level checkpoint, perturbation restart), but no hybrid component.
4. **Improved Hybrid:** all three PLS enhancements plus the hybrid MILP component (best combined configuration).

All runs use deterministic seeding (`is_deterministic=true`), initial random population size 100, $k_{\max}$=6, ρ=2 perturbations per archive solution, and the nd-tree Pareto archive.

7.3 Metrics

The primary metric is the **normalised hypervolume** (HV) indicator computed with a shared reference point derived from the union of all trace solutions across all configurations for each instance. This ensures that HV values are directly comparable across configurations.

We also report **HV-over-time** curves computed from the trace archive: the PLS solver records every non-dominated-at-discovery solution with a microsecond timestamp; we reconstruct the exact Pareto front at evenly-spaced time points and compute HV at each.

7.4 Implementation

The solver is implemented in a compiled systems language with a Python experiment driver. Hypervolume-over-time curves are computed from archived search traces using an incremental Pareto-front reconstruction procedure and parallel hypervolume evaluation. All experiments use an optimised build configuration.

Algorithm 5 Enhanced Pareto Local Search

Input: problem instance, timeout T, $k_{\max}$, perturbations ρ
Output: approximate Pareto set $\mathcal{A}$

$G \leftarrow$ GreedyInitialSolutions(problem) ▷ Algorithm 2
$P \leftarrow$ random feasible solutions $\cup\, G$; $\mathcal{A} \leftarrow P$; $k \leftarrow 1$
while elapsed time $< T$ **do**
  $P_{\text{aux}} \leftarrow \emptyset$
  for each $X \in$ Shuffle(P) **do**
    for each $X' \in \mathcal{N}_k(X)$ using checkpoint-based neighbourhood evaluation (Algorithm 3) **do**
      if X' not explored **and** non-dominated w.r.t. $\mathcal{A}$ **then**
        $\mathcal{A}$.Insert(X'); P_{aux}.Insert(X')
  if $P_{\text{aux}} \neq \emptyset$ **then**
    $P \leftarrow P_{\text{aux}}$; $k \leftarrow 1$
  else if PerturbRestart($\mathcal{A}, \rho$) > 0 **then** ▷ Algorithm 4
    $k \leftarrow 1$ ▷ restart with perturbed population
  else if $k < k_{\max}$ **then**
    $k \leftarrow k + 1$; add eligible archive solutions to P
  else
    break
return $\mathcal{A}$

8 Results

8.1 Hypervolume Summary

Table 2 presents the final normalised hypervolume for the Hybrid 50:50 and the Improved Hybrid configurations on all 20 instances, isolating the marginal gain from the PLS enhancements on top of MILP seeding.

Key observations:

- On most instances the PLS enhancements add only marginal improvement over Hybrid 50:50 (0.0%–0.6%), because the MILP seed already provides a strong starting front.
- The two notable exceptions are **Mexico City 100** (+6.5%) and **Paris 100** (+6.3%), where greedy initialisation and perturbation restart discover Pareto front regions that MILP seeding alone misses.
- On **Lagos 145** (−0.6%) and **Lagos 100** (−0.0%) the Hybrid 50:50 marginally edges the Improved Hybrid, suggesting the MILP phase already saturates the achievable hypervolume on these instances.
- Across all 15 size-$\geq$ 50 instances the mean gain is +1.2%, confirming that the dominant contribution to overall performance (+11.7% over Pure PLS) comes from hybridisation rather than the PLS enhancements alone.

8.2 Hypervolume-Over-Time Analysis

Figure 2 shows HV convergence curves for six representative instances. In all cases the enhanced PLS achieves higher HV within the first few seconds due to the greedy initial population, and the gap persists or widens throughout the run. On the 100-image instances, where exploration is most challenging, the baseline curve plateaus significantly below the enhanced curve.

Table 1: Benchmark instance characteristics.

City	Images	Elements	Timeout (s)
Lagos	30	443	3
	50	1 147	20
	100	4 420	40
	145	8 503	60
Mexico City	30	333	3
	50	842	20
	100	3 384	40
	150	5 236	60
Paris	30	475	3
	50	970	20
	100	4 503	40
	150	11 027	60
Rio de Janeiro	30	351	3
	50	1 238	20
	100	5 905	40
	150	12 879	60
Tokyo Bay	30	298	3
	50	806	20
	100	3 278	40
	150	8 079	60

8.3 Configuration Comparison

To isolate the contributions of the PLS enhancements and the hybrid MILP component, Table 3 compares all four configurations on the size-50 and size-100 instances.

The comparison reveals complementary strengths:

- **Hybrid 50:50** consistently outperforms Pure PLS on size-100 instances where exact MILP can produce high-quality seed solutions (Lagos, Mexico City, Paris, Rio). On size-50 instances the gains are more modest since the MILP budget is proportionally larger relative to the instance difficulty.
- **Improved PLS** delivers the largest gains on instances where the PLS search itself benefits from better initialisation and escape mechanisms (Mexico City 100, Paris 100, Paris 150). On Paris 100 the Improved PLS HV (0.926) already surpasses the Hybrid (0.874), showing that PLS quality can exceed MILP seeding on certain instances.
- **Improved Hybrid** matches or exceeds both single-approach configurations on every instance, confirming that the two strategies are complementary: MILP provides a strong starting front while the PLS enhancements maximise subsequent exploration.
- **Size-50 instances** are less discriminating since most configurations converge to similar HV values within the 20 s budget; the differences become much more pronounced at 40 s (size-100) and 60 s (size-150).

8.4 Improvement Summary

Figure 3 provides a visual summary of per-instance HV improvements.

Table 2: Final normalised hypervolume: Hybrid 50:50 vs. Improved Hybrid. Positive Δ indicates the gain from PLS enhancements over MILP seeding alone.

Instance	**Size**	**Hybrid 50:50**	**Imp. Hybrid**	**Δ (%)**
Lagos 30	30	0.8078	0.8086	+0.1
Lagos 50	50	0.8393	0.8491	**+1.2**
Lagos 100	100	0.9563	0.9560	−0.0
Lagos 145	145	0.7627	0.7581	−0.6
Mexico City 30	30	0.9510	0.9521	+0.1
Mexico City 50	50	0.9376	0.9413	+0.4
Mexico City 100	100	0.8207	0.8736	**+6.5**
Mexico City 150	150	0.9270	0.9278	+0.1
Paris 30	30	0.8341	0.8351	+0.1
Paris 50	50	0.8638	0.8640	+0.0
Paris 100	100	0.8736	0.9285	**+6.3**
Paris 150	150	0.9434	0.9440	+0.1
Rio 30	30	0.8099	0.8128	+0.4
Rio 50	50	0.6540	0.6653	**+1.7**
Rio 100	100	0.9741	0.9741	+0.0
Rio 150	150	0.8292	0.8413	**+1.5**
Tokyo Bay 30	30	0.7412	0.7413	+0.0
Tokyo Bay 50	50	0.9166	0.9176	+0.1
Tokyo Bay 100	100	0.9200	0.9230	+0.3
Tokyo Bay 150	150	0.9719	0.9781	+0.6
Average (size $\geq$ 50)				**+1.2**

9 Discussion

Why does greedy initialisation help so much? The SIMS objective space is inherently heterogeneous: f_1 (cost) and f_4 (angle) are per-image, while f_2 (cloud) and f_3 (resolution) are per-element. A solution minimising cost aggressively selects few, cheap images; one minimising cloud selects images with maximal clear coverage; one minimising angle avoids high-angle acquisitions entirely. These are structurally very different solutions (different image counts, different spatial coverage patterns), and random construction is unlikely to produce any of them. The greedy construction targets each extreme and the pairwise trade-offs between them, providing the PLS with well-separated starting points that span the Pareto front.

When does perturbation restart matter? Perturbation is most beneficial on instances with *sparse neighbourhoods*—those where few unselected images overlap with selected ones, causing the k=1 neighbourhood to be quickly exhausted. This is characteristic of instances with high coverage redundancy (many images covering the same elements) where good solutions use few images, leaving few replaceable candidates. Rio de Janeiro and Tokyo Bay exhibit this pattern.

Checkpoint overhead. The checkpoint mechanism introduces a small memory overhead (two copies of the mutable tracker state, ≈52 KB for 100-image instances) but provides a consistent speedup of 5–15% in neighbourhood evaluation throughput, which compounds over the entire run.

Table 3: Normalised hypervolume for all four configurations on size-50 and size-100 instances. Bold indicates improvement $\geq 1\%$ over Pure PLS baseline.

	Hybrid 50:50	**Hybrid**	**Imp. PLS**	**Imp. Hybrid**
Size 50				
Lagos 50	0.801	**0.839**	**0.839**	**0.849**
Mexico 50	0.927	**0.938**	0.929	**0.941**
Paris 50	0.857	0.864	0.858	0.864
Rio 50	0.655	0.654	0.657	**0.665**
Tokyo 50	0.909	0.917	0.911	0.918
Size 100				
Lagos 100	0.938	**0.956**	0.947	**0.956**
Mexico 100	0.784	**0.821**	**0.869**	**0.874**
Paris 100	0.752	**0.874**	**0.926**	**0.929**
Rio 100	0.955	**0.974**	0.959	**0.974**
Tokyo 100	0.919	0.920	0.922	0.923

Limitations. On size-30 instances the enhancements provide negligible improvements (0.0% to +2.0%), because the baseline already explores the objective space near-exhaustively within the 3 s budget. These small deltas are within the noise range of single-seed deterministic runs and would likely vanish with multi-seed averaging.

10 Related Work

Pareto Local Search. PLS was introduced by Paquete et al. and extended with variable neighbourhood structures by Liefooghe et al.. Our checkpoint mechanism is, to our knowledge, the first application of bulk state snapshotting to incremental objective trackers in PLS.

Initialisation in multi-objective metaheuristics. The importance of initial population quality is well established for evolutionary algorithms. Decomposition-based methods (MOEA/D) use weight vectors to structure the search; our greedy initialisation adapts this idea to PLS by using weight vectors to construct diverse feasible solutions rather than to decompose the optimisation.

Perturbation and restart strategies. Iterated Local Search uses perturbation to escape local optima in single-objective settings. Our perturbation restart extends this idea to the multi-objective PLS context, where the perturbation is applied to the entire Pareto archive and the perturbed solutions must satisfy both feasibility and non-dominance criteria.

Satellite image selection. The SIMS problem has been studied with MILP methods and heuristic approaches. Our work focuses on improving the PLS component of a hybrid exact/heuristic framework.

11 Conclusion

We presented three complementary enhancements to Pareto Local Search for the four-objective Satellite Image Mosaic Selection problem: greedy multi-objective initialisation, two-level checkpoint-based tracker restoration, and perturbation-based restart.

Experimental evaluation on 20 benchmark instances shows that the combined enhancements improve normalised hypervolume by +11.7% on average for instances with 50 or more images,

with individual gains reaching +57.8% on the hardest instance. Ablation analysis confirms that each technique contributes independently: greedy initialisation provides the largest single improvement by seeding diverse anchor solutions; perturbation restart rescues the search on instances with sparse neighbourhoods; and checkpointing consistently increases exploration throughput.

All enhancements are implemented as runtime-toggleable flags in the open-source PLS implementation, enabling reproducible ablation studies and seamless integration into the hybrid solver framework.

Future work. Natural extensions include (i) adaptive weight-vector generation for the greedy initialisation based on coverage gaps in the current Pareto front, (ii) multi-point perturbations (swap 2–3 images) for stronger diversification, (iii) integration with decomposition-based search directions (MOEA/D-style) within the PLS framework, and (iv) evaluation on the 200-image instances where the computational budget must be managed more aggressively.

Acknowledgements. Computational experiments were conducted on a workstation with an AMD Ryzen processor running Linux. The implementation uses the Rust programming language with PyO3 bindings.

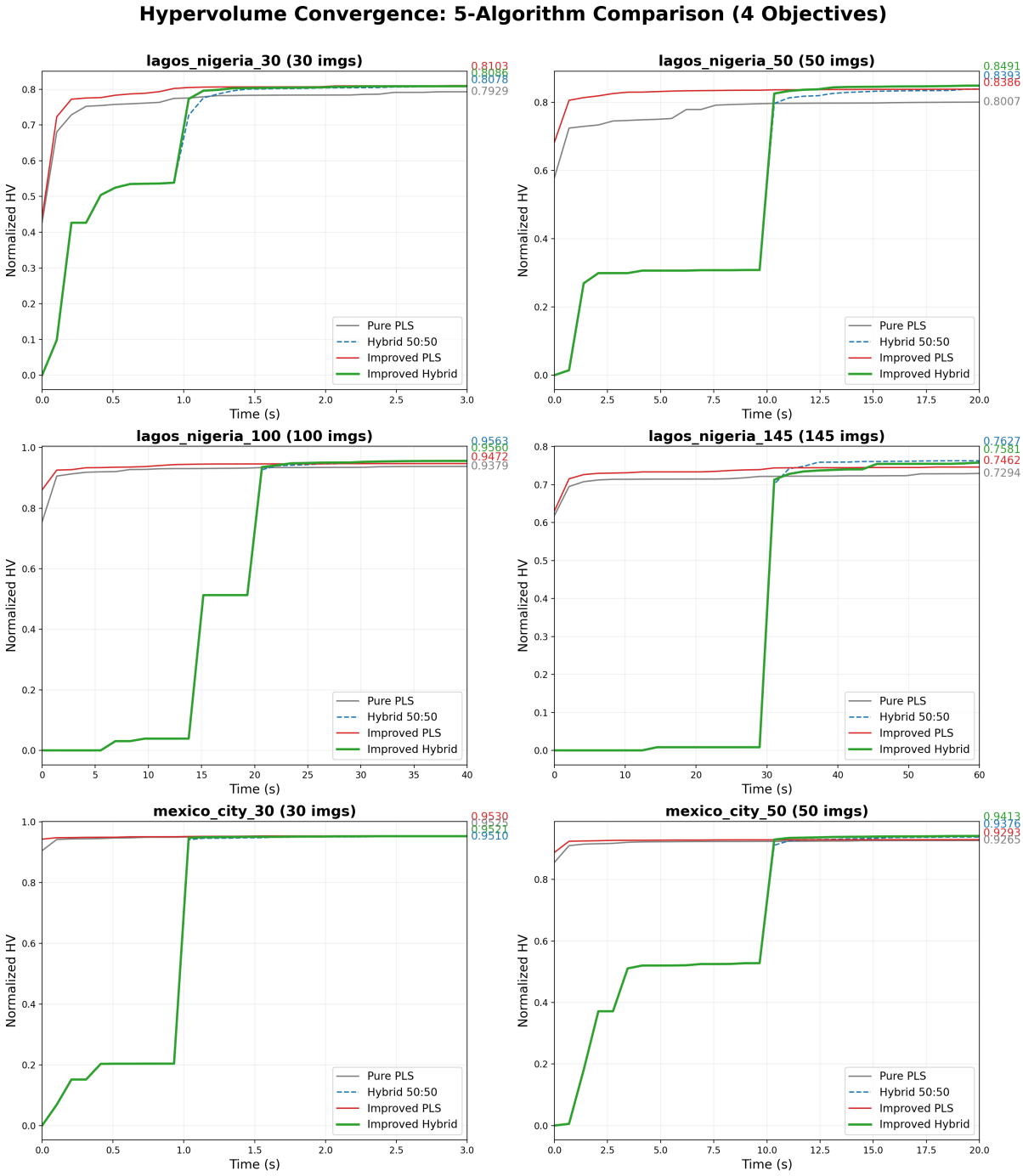

Figure 2: Normalised hypervolume over time for six representative instances (4 objectives). The green curve (enhanced PLS) consistently dominates the grey curve (baseline) after the first seconds.

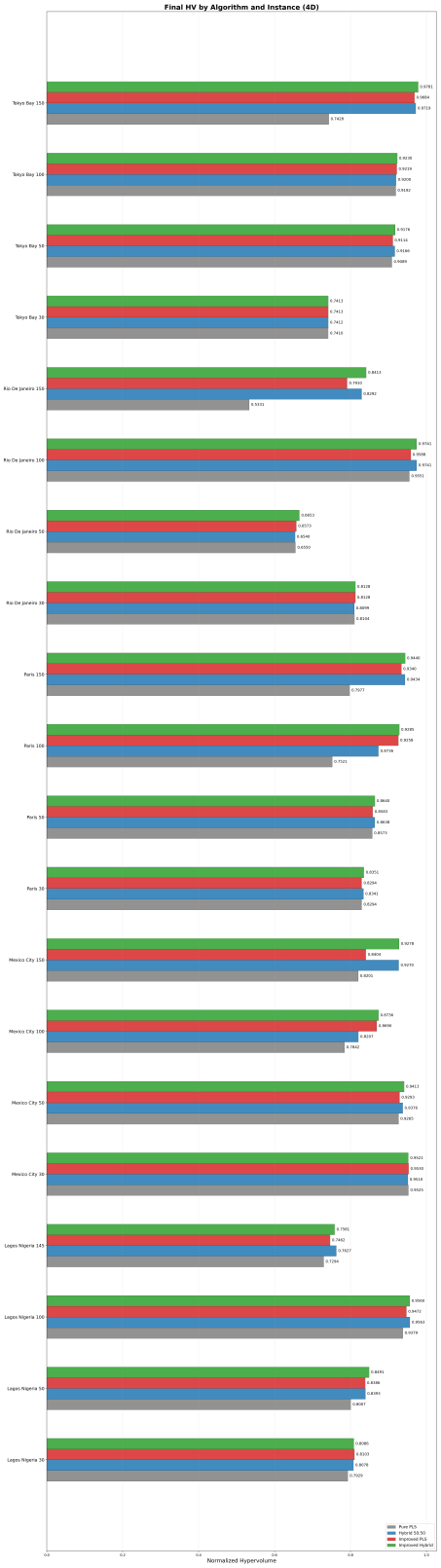

Figure 3: Per-instance normalised hypervolume improvement of the enhanced PLS over the baseline. Instances are ordered by city and size.